

BANQUE MISR

Vectorless Retail Banking RAG Assistant

Technical Architecture & Design Report

This document explains the complete system end to end: what problem it solves, why a vectorless architecture was chosen over conventional embedding-based RAG, how the system evolved from Phase 1 (a working baseline) into Phase 2 (a full agentic pipeline with query understanding, corrective retrieval, and dual safety guardrails), and exactly what happens, component by component, when a customer asks a question. It is written so that someone with no prior context on the project can read it and present the system confidently.

1. Executive Summary

The system is a conversational assistant that answers customer questions about Banque Misr's credit cards, merchant offers, and campaigns, grounded entirely in the bank's own structured data — never in the model's general knowledge for product-specific facts. It runs entirely on local infrastructure (Ollama, serving open-weight models), which means no customer data or query ever leaves the bank's own machines.

The defining architectural choice is that it uses no vector database and no embeddings at all. Instead, it builds a navigable tree index over the bank's records and lets a language model read and traverse that tree directly to decide what's relevant — the same way a person would scan a table of contents rather than search by semantic similarity. This is what "vectorless RAG" means in this project's name.

On top of that retrieval foundation, the system adds a full agentic control layer: it understands multi-turn conversations, decides for itself whether and where to search, corrects its own retrieval when the first attempt comes back weak, and checks its own final answer for fabricated facts, inappropriate financial advice, or accidental disclosure of internal system details before anything reaches the customer.

2. Why Vectorless RAG

Conventional RAG systems chop documents into chunks, embed each chunk as a vector, and retrieve by nearest-neighbour similarity search. That approach has a specific weakness that matters a great deal for structured banking data: similarity search finds text that sounds like the question, not text that is the entity the question is actually about.

The concrete failure mode this avoids

Asked "What installment offers does Carrefour have?", a similarity search over offer chunks will often surface OTHER merchants' installment offers too, because "installment offer" is the phrase driving the similarity match, not the merchant name. The system then has to somehow filter that out after the fact, and a generator shown three merchants' offers side by side will sometimes blend them.

This project's data — credit cards, merchant offers, campaigns — already has stable, human-readable identity: a card has a name, an offer belongs to exactly one merchant, a campaign has a defined scope. That structure is worth preserving rather than dissolving into embedding space. So instead of embeddings, the system:

1. Renders every record as clearly labelled markdown, organised under headings by category.
2. Builds a hierarchical tree from that markdown, where each heading becomes a navigable node.
3. Lets a language model read the tree's structure directly and decide which branches are worth descending into for a given question — an actual reasoning step, not a distance calculation.

A useful way to describe the difference to someone unfamiliar with either approach: a vector database finds the paragraph that sounds most like your question. A vectorless tree lets the model read the table of contents and decide where to look, the way a person would. The trade-off is that vectorless retrieval requires more from the model at query time — it has to reason about the tree, not just compute a distance — but it produces results that respect entity boundaries (merchant, card, campaign) far more reliably, which matters more in banking than in most other RAG applications.

A second, practical advantage: because records keep stable IDs (e.g. card_303, offer_installment_4_Carrefour), retrieval quality can be measured directly and objectively — precision, recall, and F1 against a hand-labelled gold set of correct record IDs per test question — without needing an LLM judge or embeddings to define what "correct" even means. This is what the project's Tier 1 evaluation (Section 8) is built on, and it is a benefit specific to structured, ID-addressable data that a chunk-and-embed system does not get for free.

3. Phase 1 — The Retrieval Foundation

Phase 1 built and validated the core retrieval mechanism before any conversational intelligence was added on top. Its job was narrow and specific: given a question, find the right records, reliably.

3.1 Indexing — turning records into a navigable tree

Each data source (cards, offers, campaigns) goes through an indexing pipeline that runs once, offline, before the assistant ever answers a live question:

1

Categorize

A language model assigns each record a short category label (e.g. "Credit Cards"), used purely to organize the tree — this is the only LLM step in indexing besides the optional summary step below.

2

Render to markdown

Every record is written out as a markdown section under its category heading, with a consistent, predictable structure the tree builder can parse deterministically.

3

Build the tree (PageIndex)

An open-source library (PageIndex) converts the markdown's heading structure directly into a tree of nodes — this step is fully deterministic (no LLM), so the hierarchy can never come out malformed the way an earlier hand-rolled, LLM-generated-JSON tree builder sometimes did.

4

Persist

The finished tree is saved to disk (TreeStorage) as the system's index for that data source, to be loaded once at server startup and queried many times.

A deliberate, well-documented design decision here: node summaries (an LLM-written one-line description per tree node, to help the traversal model navigate) are OFF by default. The underlying library has no way to limit how many summary requests it fires at the local model concurrently, and under load this caused every single request to individually time out waiting its turn — a measured, reproduced failure, not a theoretical one. The tree remains fully usable without summaries; node titles and record IDs alone are enough for the traversal model to navigate correctly.

3.2 Retrieval — the tree traverser

At query time, a language model is shown the tree's structure (titles, IDs, and summaries where present) and asked which nodes are relevant to the question, returning a JSON list of selected node IDs. Critically, the returned IDs are validated in Python against the real tree before being trusted — an invented ID is simply dropped rather than causing a crash or a fabricated lookup. This whitelist-and-validate pattern ("the model proposes, Python disposes") recurs throughout the whole system and is one of its core engineering principles; Section 7.1 covers it in full.

3.3 The generator

The retrieved records are formatted into a labelled context block and handed to a final language model along with the question, under a system prompt that explicitly forbids inventing any card name, merchant name, fee, or date not present in the retrieved text, and requires the model to say plainly when the data doesn't answer the question rather than guess.

4. Phase 2 — The Agentic Control Layer

Phase 1 answered one question at a time, with no memory, no judgement about whether retrieval was even necessary, and no verification of its own output. Phase 2 adds four capability areas on top of that same retrieval foundation, none of which required rewriting the retriever itself — a deliberate benefit of keeping retrieval and everything around it as separate, replaceable stages from the start.

Field	Detail
Query Understanding & Control	Resolves multi-turn context, decides IF and WHERE to search, and screens incoming messages before they reach anything else.
Corrective Retrieval (CRAG)	Grades what retrieval actually found, re-ranks it, and automatically widens the search once if the first attempt came back weak.
Output Safety	Checks the generated answer for fabricated facts, personalised financial advice, or leaked internal details before it reaches the customer.
Observability & Evaluation	Every pipeline stage logs its decision to a structured, queryable log, and a dedicated evaluation harness measures retrieval quality objectively.

5. Query Understanding & Retrieval Control

5.1 Conversation Memory

Each active conversation (identified by a `session_id` generated client-side and stored in the browser) keeps a small rolling window of the most recent raw turns, verbatim. When a turn falls out of that window, it is not simply discarded — it is folded into a running plain-text summary via a dedicated LLM call, which explicitly receives the current summary alongside the turn being folded in, so older context accumulates rather than being overwritten each time a new turn rolls off.

This LLM call runs lazily — only when a turn is actually about to be evicted from the raw window — rather than on every single message, which would add an unnecessary model call to every short exchange for no benefit.

5.2 Query Contextualizer

Before anything else happens, the customer's raw message is rewritten into a standalone question using the conversation memory above. "What about its annual fee?" becomes "What is the annual fee on the VISA Infinite card?" — resolving the pronoun using what was actually discussed. Every downstream stage (the router, the retrievers) then works from this resolved question, never from the ambiguous original.

If there is no prior conversation to resolve against, this step is skipped entirely rather than needlessly calling the model on a question that is already standalone. If the rewrite ever fails or comes back empty, the system falls back to the original raw question rather than passing an empty string downstream — a small instance of the same never-trust-the-model's-output discipline.

5.3 Router

Phase 1 queried every single data source on every question. The router replaces that with an actual decision: does this question need retrieval at all (a greeting, or a general "what is APR"-style question, does not), and if so, which of cards, offers, or campaigns are actually relevant.

The router's raw output is never trusted directly. Every source name it proposes is checked against a fixed whitelist in Python, and if the whole response is unusable — malformed JSON, wrong types, or every proposed source turning out to be invalid — the system deliberately falls back to querying every source, exactly as Phase 1 always did. Falling back

to "search everything" was chosen over "search nothing", because a failed router call that resulted in zero retrieval would look identical to a real "no relevant data exists" case, silently producing a worse answer with no visible error to investigate.

5.4 Input Guardrail

The very first thing that happens to any incoming message — before contextualization, before routing — is a safety classification into one of four categories: safe, off-topic, an injection attempt (trying to override the assistant's instructions or extract its system prompt), or a sensitive request (asking for something the bot must never provide, such as another customer's details). Anything other than "safe" short-circuits the entire rest of the pipeline and returns a canned response immediately — contextualization, routing, and retrieval never even run for a blocked message.

This check deliberately runs on the RAW message, before the contextualizer touches it, because a rewrite aimed at producing a clean standalone question could accidentally launder an injection attempt into something that reads as an innocent question by the time a later stage would see it.

If the classification call itself fails or returns something unusable, the system fails open — the message is allowed through, logged with a visible `fail_open` flag. This was a deliberate, discussed trade-off for this project's scope: failing closed would mean a single bad model response makes the entire assistant unusable, which is a worse outcome for a demo banking assistant with no real customer data behind it than the small risk of one unclassified message getting through.

6. Corrective Retrieval (CRAG)

Retrieval is not assumed to be correct just because it returned something. After the router's chosen sources are queried, every returned record is graded by a dedicated LLM call against three categories: relevant, partially relevant, or irrelevant to the actual question — with an explicit "named-entity rule" in its instructions (a different merchant's installment offer is irrelevant even though it is the same kind of thing as the one asked about, which is the single most common mistake a smaller local model makes on this task).

The same call also produces a ranking, so grading and re-ranking are done together in one pass rather than two separate model calls — both tasks depend on the same underlying judgement ("how useful is this record for this question"), so asking twice would double latency for no accuracy benefit.

The corrective branch

If nothing the router originally searched came back genuinely relevant, the system does not simply accept a weak answer. It widens the search to the sources the router chose NOT to consult, re-grades the combined result, and — if even that comes back weak — tells the generator explicitly that the available data is incomplete, rather than letting it write confidently from a poor partial match. This is the corrective step CRAG is named for.

One explicit and important scope decision: this project's version of CRAG never reaches out to the open web, unlike the technique's original academic formulation. Pulling in unvetted external content for a banking assistant is not meaningfully different from letting the model invent facts, and it would directly undermine the grounded-answers

requirement the whole system is built around. "Widen the search" here always means searching more of the bank's own vetted internal sources, never the internet — and the retry is capped at exactly one attempt, which bounds worst-case latency on a system where every step is a local model call.

7. Output Safety

The last checkpoint before any answer reaches a customer reviews the generated text against the context it was supposed to be built from, checking for three distinct problems, each handled with a different severity because they are not equally serious:

Field	Detail
Fabrication	A specific factual claim (a fee, rate, date, product name) that does not appear anywhere in the retrieved context. Annotated by default, not blocked — the checker is itself a small local model prone to false alarms, and silently withholding a correct answer is its own failure mode. A configuration flag exists to block instead, for deployments that prefer silence to any risk.
Personal financial advice	Telling this specific customer what they personally should do with their money, as opposed to describing or comparing products. Always annotated with a standard disclaimer, never blocked — the line between explaining a product and "recommending" it is genuinely blurry, and an over-eager guardrail that blocks legitimate product explanations is a worse product than one that adds a caveat.
Internal disclosure	The answer reveals system prompts, model names, or internal architecture. Always blocked outright — there is no legitimate version of this happening.

As with every other model-driven decision in this system, the reviewer's raw output (a set of booleans and short quoted claims) is never allowed to directly determine what the customer sees. Python decides the final action, chooses the replacement text where needed, and applies a fixed severity order — disclosure outranks fabrication, which outranks advice — so that the most serious finding's response wins rather than several different caveats being stacked on top of each other.

8. Full Request Lifecycle — One Question, Start to Finish

This section traces exactly what happens, in order, when a customer sends a single message. Every step below is a real, separate stage in the code, and every stage writes a structured log entry (Section 9) recording what it decided.

1	Message arrives The browser sends the raw message plus a persistent <code>session_id</code> (generated once in the browser and stored in <code>localStorage</code>) to the server.
---	---

2

Input Guardrail

The RAW message is classified as safe / off-topic / injection attempt / sensitive request. If not safe, the pipeline stops here and a canned response is returned immediately.

3

Conversation Memory read

The session's rolling summary and recent raw turns are retrieved (empty if this is the first message).

4

Query Contextualizer

The raw message is rewritten into a standalone question using that memory, resolving any references to earlier turns. Skipped entirely if there is no prior context to resolve.

5

Router

Decides whether this standalone question needs any data retrieval at all, and if so, which of cards / offers / campaigns to search.

6

Retrieval

The tree retriever for each chosen source is queried; a language model reads that source's tree and selects the relevant node IDs, which are validated against the real tree before being trusted.

7

Crag Grading

Every retrieved record is labelled relevant / partially relevant / irrelevant and ranked. If nothing came back genuinely relevant, the search widens once to the unsearched sources and is re-graded.

8

Context Building

The surviving, ranked records are formatted into one labelled text block, with each record's length adaptively capped so that however many records survived, the total never overflows the model's context window.

9

Generation

The context block and the question are sent to the generator model under a strict system prompt: answer only from the retrieved information, or say plainly that the data doesn't cover it.

10

Output Guardrail

The draft answer is checked against the same context for fabrication, personal advice, or internal disclosure, and the action (allow / annotate / block) is applied to produce the final text.

11

Memory write & response

The FINAL text (post-guardrail, not the draft) is saved into conversation memory, and that same final text is returned to the customer.

9. Cross-Cutting Engineering Principles

9.1 "The model proposes, Python disposes"

This is the single most repeated discipline in the codebase, applied identically at every stage that asks a language model for a structured decision: the tree traverser's selected node IDs, the router's chosen sources, the grader's relevance labels, and the output guardrail's flagged claims. In every case, the raw model output is parsed defensively, checked against a fixed set of valid values in Python, and anything invalid is dropped or triggers a safe, predefined fallback — never trusted or passed through as-is. This is what makes a system built on small, occasionally-unreliable local models behave predictably in aggregate.

9.2 The context-window truncation lesson

Ollama, the local model runtime this project uses, does not default to a model's full context window — unless a role's context size is explicitly configured, it silently truncates any longer prompt with no error or warning at all. This caused two separate, measured production bugs during development: a tree-traversal prompt was cut to roughly 15% of its real size, causing every question to retrieve the same handful of records regardless of what was actually asked; and later, a generator prompt was silently cut mid-record, causing the model to state a card's annual fee as a number that appeared nowhere in that card's actual data.

The fix in both cases was the same principle applied at two different points: every model role in the system has its own explicitly-configured context size, sized to what that role genuinely sends rather than copied from a neighbouring role's number (copying a number was literally how the second bug happened); and, on the content side, the total amount of retrieved text sent to the generator is capped by a shared, adaptive budget — divided across however many records survived grading, with a floor beneath which a record would become too short to be useful — so the total is guaranteed never to overflow the window no matter how many or how few records were kept.

9.3 Structured, queryable observability

Every pipeline stage — the input guardrail, the contextualizer, the router, retrieval, CRAG grading and any retry, generation, and the output guardrail — writes one structured JSON log entry per decision, tagged with a shared session and turn identifier so every stage belonging to one customer message can be reconstructed and correlated afterward. This was treated as a first-class piece of infrastructure built before any of the components that depend on it, specifically so that when something in this many-stage, multi-model pipeline behaves unexpectedly, the actual sequence of decisions that produced it is visible rather than having to be guessed at from the final answer alone.

10. Testing & Evaluation

The system is checked at two distinct levels, deliberately kept separate because they answer different questions.

10.1 Unit and integration testing

A dedicated test suite exercises every Phase 2 component's logic in isolation — parsing, validation, and every documented fallback path (malformed model output, an unreachable model, a hallucinated source or verdict) —

using mocked model responses so this suite runs in under a second and never depends on Ollama being available. Layered on top of that, a small set of live end-to-end tests exercise the fully wired pipeline against a real running model, confirming not just that each piece works alone but that the actual request lifecycle described in Section 8 behaves correctly together — including verifying, from the structured logs the run produces, that a genuine injection attempt is correctly blocked before any other stage runs, and that a genuine follow-up question is correctly resolved using conversation memory.

10.2 Retrieval evaluation (Tier 1)

A separate, deterministic evaluation harness measures retrieval quality directly: a hand-labelled test set defines the correct record IDs for a range of questions, and the harness reports precision, recall, and F1 of what retrieval actually returned against that gold set — optionally with CRAG grading applied on top, to measure whether grading measurably improves precision and whether the corrective retry fires exactly where it should. As Section 2 notes, this kind of objective, LLM-judge-free evaluation is only possible because the underlying data has stable, addressable record identity — a direct benefit of the vectorless architecture.

11. Deliberate Scope Decisions

A few choices in this system were made consciously, with a documented trade-off, rather than being oversights — worth being able to explain if asked:

- CRAG never queries the open web, only the bank's own other data sources — see Section 6.
- The input guardrail fails open (allows a message through) rather than closed, if its own model call fails — see Section 5.4.
- Fabrication in the output guardrail is annotated, not blocked, by default — see Section 7.
- Conversation memory and session state are held in a single process's memory, not a database — acceptable for this project's scope, meaning state resets if the server restarts.
- PageIndex node summaries are off by default due to a measured, unbounded-concurrency limitation in that library, not a limitation of the tree structure itself — see Section 3.1.

End of report.